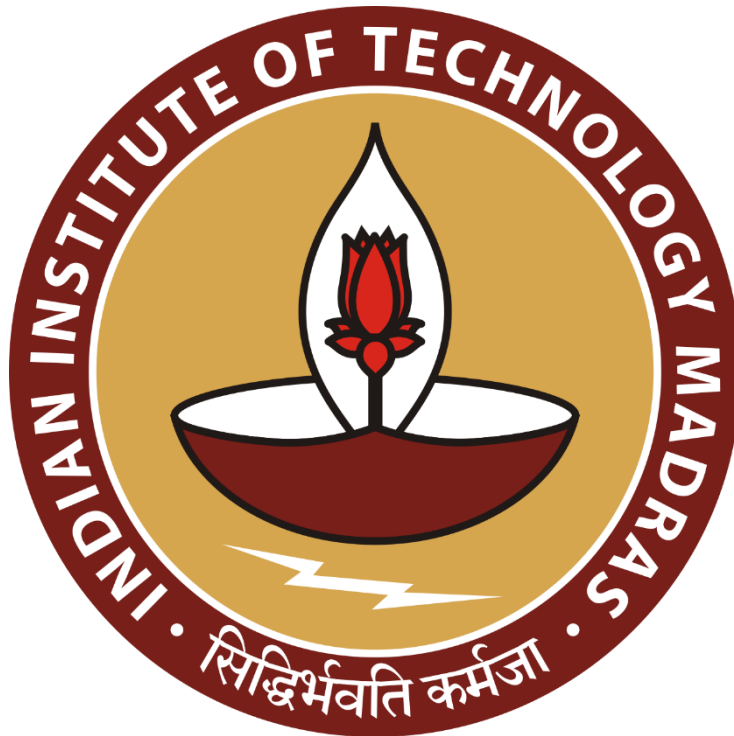




Milestone 3 Report: Context Augmentation with RAG Pipelines



JULY 10, 2026
NAME: JOB BENNY NAMBERIL
Roll Number: 22f3000174

Project Report

Milestone 3: Context Augmentation with RAG Pipelines

Student

Name: JOB BENNY NAMBERIL

Course

Deep Learning – Milestone 3

Project Title

Context Augmentation with Retrieval-Augmented Generation (RAG) Pipelines

1. Introduction

Large Language Models (LLMs) possess extensive pre-trained knowledge but are limited to the information learned during training. They cannot reliably access newly available information or large private knowledge bases without additional mechanisms. Retrieval-Augmented Generation (RAG) addresses this limitation by retrieving relevant external information and supplying it to the language model before inference.

The objective of this milestone was to build and evaluate a complete RAG pipeline consisting of knowledge base creation, dense retrieval using FAISS, semantic reranking using a Cross-Encoder, prompt augmentation, and downstream reasoning using a zero-shot classifier.

2. Objectives

The primary objectives of this milestone were:

- Understand the limitations of standalone LLMs.
 - Build a knowledge base from the training dataset.
 - Generate dense sentence embeddings using Sentence Transformers.
 - Index embeddings using FAISS.
 - Retrieve relevant documents for a query.
 - Improve retrieval quality through Cross-Encoder reranking.
 - Augment prompts with retrieved context.
 - Measure retrieval quality using Hit Rate.
 - Evaluate prediction quality using MAP@3.
-

3. Technologies Used

- Python

- Pandas
 - NumPy
 - FAISS
 - Sentence Transformers
 - CrossEncoder
 - Hugging Face Transformers
 - Facebook BART Large MNLI
 - BERT Base Uncased Tokenizer
 - Scikit-Learn
-

4. Models Used

Embedding Model

Model:

all-MiniLM-L6-v2

Purpose:

- Convert documents into dense vector embeddings.
 - Convert user prompts into embeddings.
 - Enable semantic similarity search using FAISS.
-

Retrieval Engine

FAISS IndexFlatL2

Purpose:

- Store document embeddings.
 - Perform nearest-neighbor search.
 - Retrieve the Top-K most relevant documents.
-

Cross Encoder

Model:

cross-encoder/ms-marco-MiniLM-L-6-v2

Purpose:

- Score each retrieved document against the query.
- Improve ranking accuracy.

- Select the most semantically relevant document.
-

Zero-Shot Classifier

Model:

facebook/bart-large-mnli

Purpose:

- Predict the correct multiple-choice option.
 - Produce probability scores for all candidate answers.
-

5. Knowledge Base Construction

The training dataset was loaded into memory.

For every row:

1. The correct answer letter was identified.
2. The corresponding answer text was extracted.
3. The text was appended to the Knowledge Base.

Example:

Answer = C

Knowledge Base Document

"The butterfly effect is the phenomenon that a small change in the initial conditions..."

Every knowledge base document was embedded using:

SentenceTransformer("all-MiniLM-L6-v2")

The embeddings were indexed using FAISS.

6. Retrieval Pipeline

For every prompt:

1. Encode the prompt.
2. Search the FAISS index.
3. Retrieve the Top-K documents.

FAISS performs retrieval using dense vector similarity.

Advantages:

- Extremely fast
- Scales to millions of documents

Limitation:

- Retrieval is based solely on embedding similarity.
 - The most relevant document is not always ranked first.
-

7. Cross-Encoder Reranking

To improve retrieval quality, the retrieved documents were passed through a Cross-Encoder.

Each query-document pair was processed jointly.

Example:

Question + Document 1

↓

Cross Encoder Score

The document with the highest semantic relevance score was selected.

Compared to FAISS, Cross-Encoders are slower but considerably more accurate because the Transformer jointly attends to both the question and document.

8. Retrieval-Augmented Generation (RAG)

After reranking, the highest-ranked document was injected into the prompt.

The final prompt format was:

Context: [Retrieved Document]

Question: [Original Prompt]

This augmented prompt was then passed to the BART zero-shot classifier for prediction.

9. Experimental Tasks

The milestone included several experiments.

Experiment 1

Run a zero-shot classifier without external context and record the probability assigned to the correct answer.

Experiment 2

Retrieve documents using FAISS.

Measure the ranking position of the correct document.

Experiment 3

Apply Cross-Encoder reranking.

Compare the new ranking with FAISS.

Experiment 4

Measure the total number of BERT tokens after augmenting the prompt with retrieved context.

Experiment 5

Inject the true document into the prompt.

Observe the improvement in the confidence of the correct answer.

Experiment 6

Perform Adversarial RAG.

Inject an unrelated document as context.

Observe how incorrect retrieval negatively affects prediction confidence.

This experiment demonstrated the principle:

Garbage In → Garbage Out

Experiment 7

Evaluate retrieval quality.

Metric used:

Hit Rate@5

A retrieval counts as a hit when the correct document appears within the Top-5 retrieved documents.

Experiment 8

Construct a complete RAG pipeline:

- Retrieve
- Rerank
- Augment
- Predict
- Evaluate

Performance was measured using:

MAP@3

This metric rewards correct answers appearing near the top of the predicted ranking.

10. Pipeline Architecture

Question

|



Sentence Embedding

|



FAISS Retrieval

|

Top-5 Documents

|



Cross Encoder

|

Best Document

|



Prompt Augmentation

|



facebook/bart-large-mnli

|



Predicted Answer

|



Evaluation (MAP@3)

11. Evaluation Metrics

Hit Rate

Measures retrieval effectiveness.

Formula:

Hits / Total Queries

Higher values indicate better retrieval performance.

MAP@3

Mean Average Precision at 3 evaluates ranking quality.

A prediction receives:

- 1.0 if correct at Rank 1
- 0.5 if correct at Rank 2
- 0.333 if correct at Rank 3
- 0 otherwise

The average across all evaluated samples gives the final MAP@3 score.

12. Key Learnings

During this milestone, the following concepts were learned:

- Building dense vector knowledge bases.
- Sentence embeddings using MiniLM.
- Similarity search with FAISS.
- Limitations of Bi-Encoder retrieval.
- Cross-Encoder reranking.
- Prompt augmentation.
- Retrieval-Augmented Generation (RAG).
- Adversarial retrieval and its impact.
- Retrieval evaluation using Hit Rate.

- End-to-end evaluation using MAP@3.
-

13. Conclusion

This milestone successfully demonstrated the complete Retrieval-Augmented Generation workflow. A knowledge base was created from the training data, indexed with FAISS, and queried using dense embeddings. Retrieval quality was improved through Cross-Encoder reranking, and the highest-ranked document was incorporated into the prompt before zero-shot classification. Additional experiments showed that providing relevant context increases model confidence, while irrelevant context can significantly degrade performance. Finally, the overall pipeline was evaluated using Hit Rate and MAP@3, illustrating the effectiveness of combining retrieval, reranking, and language models to improve question-answering performance.